Compte-rendu pédagogique — Plateforme d'analyse d'images médicales

> *Document de préparation à la soutenance. Tout est expliqué de zéro : après lecture, tu dois pouvoir répondre à n'importe quelle question technique sur le projet.*

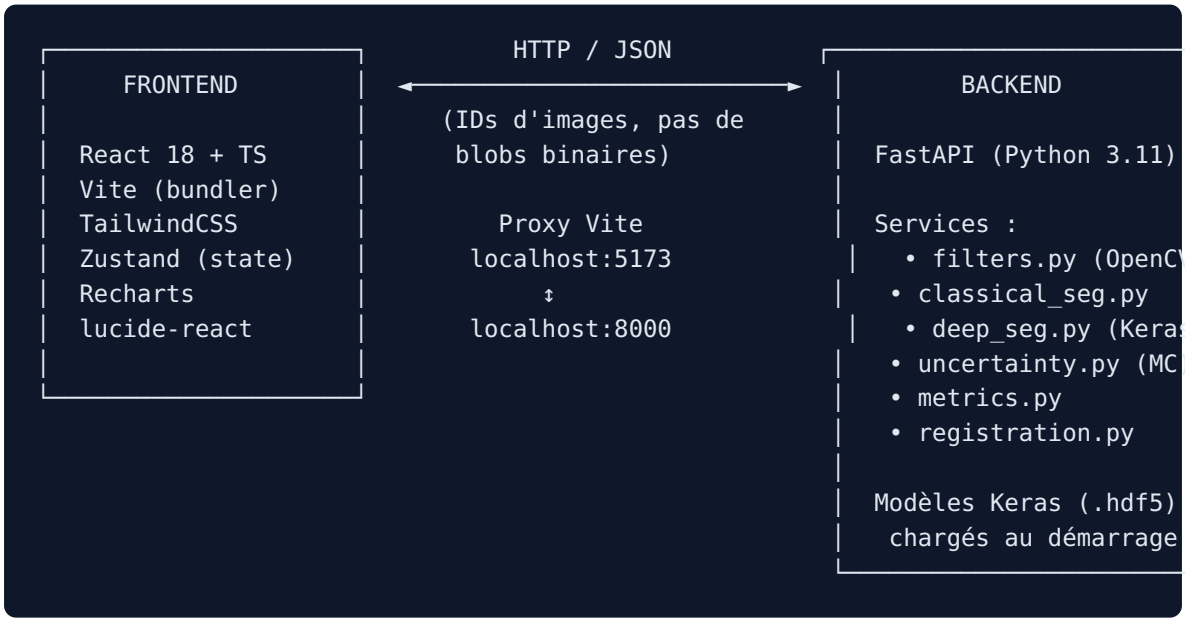
---

1. Vue d'ensemble

Ce que fait la plateforme

La plateforme est une application web qui permet à un utilisateur d'importer une image médicale (endoscopie ou IRM cérébrale), de lui appliquer des traitements (filtrage, amélioration de contraste, opérations morphologiques), puis de la segmenter — c'est-à-dire de séparer automatiquement la zone d'intérêt (tumeur, polype) du reste de l'image — par des méthodes classiques ou par deep learning. Elle produit aussi, grâce au Monte Carlo Dropout, une carte d'incertitude qui dit **où** le modèle hésite et un score global de confiance qui dit **si** on peut se fier à la prédiction.

Architecture



Les 3 piliers du cahier des charges et comment ils sont satisfaits

Pilier du cahier des charges	Comment il est satisfait
Prétraitement (filtrage, contraste, bruit)	9 méthodes dans `filters.py` + 5 opérations morphologiques + 3 détecteurs de contours, tous exposés via API REST et accessibles dans la sidebar du Workspace
Segmentation conventionnelle	7 méthodes (Otsu, adaptatif, watershed, Chan-Vese, K-means, Mean Shift, region growing) avec évaluation Dice/IoU
Segmentation deep learning pré-entraîné	U-Net (architecture TP4) entraînable sur endoscopie et IRM, chargé au démarrage, accessible en un clic

Valeur ajoutée au-delà du cahier des charges

Le module de **quantification d'incertitude par Monte Carlo Dropout** n'est pas demandé par la prof. C'est une extension recherche qui :
- Exploite les couches Dropout **déjà présentes** dans le U-Net (aucun ré-entraînement)
- Produit une carte d'incertitude pixel par pixel et un score de confiance global
- Est validée par une mini-étude benchmarkée (notebook avec 4 expériences)
- Apporte une dimension « aide à la décision clinique » au projet

---

2. Architecture technique

2.1 Backend FastAPI

Pourquoi FastAPI et pas Flask

Critère	Flask	FastAPI
Performances	Synchrone, un worker bloqué par requête	Asynchrone natif (ASGI), gère la concurrence
Typage	Pas de validation intégrée	Pydantic valide automatiquement les payloads JSON
Documentation	Manuelle (Swagger à configurer)	Swagger UI auto-générée sur `/docs`
Modernité	Python 2 legacy, callbacks	Python 3.6+ natif, type hints

Concrètement : quand on envoie `{"image_id": "abc123", "params": {"sigma": 2.0}}`, FastAPI vérifie automatiquement que `image_id` est un string et que `sigma` est un nombre. Avec Flask, il faudrait écrire cette validation à la main.

Le lifespan et le chargement des modèles

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    # Au démarrage : charger les modèles UNE SEULE FOIS
    MODELS["endoscopy"] = load_model("unet_endoscopy.hdf5", compile=False)
    yield  # L'application tourne, les requêtes arrivent
    MODELS.clear()  # À l'arrêt : libérer la mémoire
```

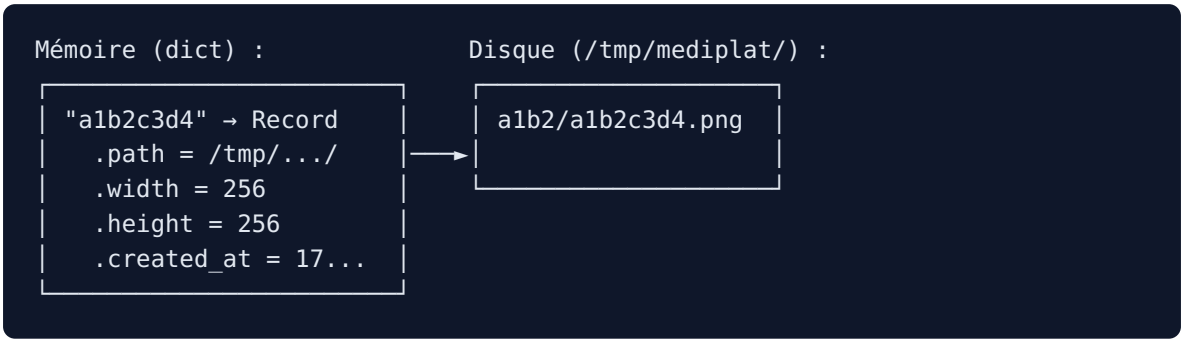
Pourquoi c'est critique : charger un modèle Keras prend 2-5 secondes (lecture du fichier HDF5, reconstruction du graphe de calcul, allocation GPU/CPU). Si on le chargeait à chaque requête, chaque segmentation prendrait 5+ secondes rien qu'en chargement. Avec le lifespan, le modèle est chargé une seule fois au démarrage et reste en mémoire pour toutes les requêtes.

Le `compile=False` évite de recompiler l'optimiseur (Adam, etc.) puisqu'on ne va pas entraîner — seulement faire de l'inférence.

Si un fichier `.hdf5` est absent, on logue un warning et on continue. Les routes classiques marchent toujours ; seule la route DL correspondante renvoie une erreur 503.

Comment fonctionne `image_store`

L'image store est un registre en mémoire (un dictionnaire Python) qui associe un **UUID** (identifiant unique) à un **chemin de fichier** sur le disque.



- `allocate_path(suffix)` : génère un nouvel UUID, crée le sous-dossier (les 4 premiers caractères servent de shard pour éviter d'avoir 10 000 fichiers dans un seul dossier), et renvoie le chemin
- `save(image_id, path, w, h)` : enregistre l'entrée dans le dictionnaire
- `get(image_id)` : cherche l'entrée, vérifie que le fichier existe encore sur le disque
- `cleanup_expired()` : supprime les entrées plus vieilles que 1 heure (TTL)

Pourquoi des IDs et pas des fichiers temporaires : parce que chaque étape du pipeline (upload → CLAHE → Otsu → évaluation) produit une image intermédiaire. On a besoin de pouvoir référencer n'importe laquelle à tout moment (pour revenir en arrière, pour comparer, pour évaluer).

Trajet d'une requête

Prenons l'exemple : « l'utilisateur applique un filtre CLAHE sur une image ».

```
1. Frontend : POST /api/preprocess/clahe
   Body: {"image_id": "a1b2c3d4", "params": {"clip_limit": 3.0}}

2. Router (preprocessing.py) :
   - Valide le JSON via Pydantic (PreprocessRequest)
   - Vérifie que "clahe" existe dans METHODS
   - Récupère le Record depuis image_store.get("a1b2c3d4")
   - Charge les pixels depuis le disque : load_image(rec.path) → numpy array

3. Service (filters.py) :
   - clahe(image, clip_limit=3.0) → nouveau numpy array

4. Retour au router :
   - Alloue un nouvel ID : image_store.allocate_path(".png")
   - Sauvegarde le résultat sur disque : save_as_png(result, path)
   - Enregistre dans le store : image_store.save(result_id, path, w, h)
   - Renvoie {"result_id": "e5f6g7h8", "duration_ms": 12.3}

5. Frontend :
   - Reçoit le result_id
   - Met à jour le state Zustand (currentResultId = "e5f6g7h8")
   - Affiche l'image via <img src="/api/image/e5f6g7h8" />
```

2.2 Frontend React

Pourquoi Vite + TypeScript + Tailwind

- **Vite** : bundler ultra-rapide (Hot Module Replacement en <50ms vs 2-3s avec Webpack). En dev, il sert les modules ES natifs sans bundling, ce qui accélère le rechargement.
- **TypeScript** : détecte les erreurs de type à la compilation. Quand l'API renvoie `{mask_id: string}` et que le frontend attend `{maskId: string}`, TypeScript le signale immédiatement.
- **TailwindCSS** : classes utilitaires (`px-4 text-sm border-border`) au lieu de CSS custom. Permet un style cohérent sans fichier CSS séparé.

Zustand vs Redux

Zustand est un gestionnaire d'état minimaliste. Comparaison :

Aspect	Redux	Zustand
Boilerplate	Actions, reducers, dispatch, connect	Une seule fonction `create()`
Taille	~10 Ko	~1 Ko
Complexité	Store global + middleware	Hook direct
Notre besoin	Trop lourd pour ce projet	Parfait : un seul store avec ~15 champs

Notre store contient : `imageId`, `currentResultId`, `maskId`, `groundTruthId`, `modality`, `activeTab`, `pipeline` (historique des étapes), `loading`, `uncertaintyData`, `metrics`.

Flux d'une interaction utilisateur

L'utilisateur clique sur « CLAHE » dans la sidebar :

```
1. onClick → handlePreprocess("clahe")
2. setLoading(true) → le bouton devient grisé, la barre de progression apparaît
3. await preprocess("clahe", currentResultId, {clip_limit: 3.0})
   → POST /api/preprocess/clahe (via Axios, proxied par Vite)
4. Réponse : {result_id: "e5f6g7h8", duration_ms: 12.3}
5. setCurrentResult("e5f6g7h8") → l'onglet "Prétraité" affiche la nouvelle image
6. setActiveTab("preprocessed") → l'onglet s'active automatiquement
7. addStep({type: "preprocess", method: "clahe", ...}) → apparaît dans l'historique à droite
8. setLoading(false) → la barre disparaît
```

2.3 Communication frontend/backend

Pourquoi des IDs et pas du base64

Une image médicale de 256×256 en niveaux de gris pèse ~65 Ko en PNG. En base64, elle pèserait ~87 Ko (overhead de 33%). Pour une image DICOM haute résolution (2048×2048), ça peut dépasser 10 Mo en base64.

Avec les IDs :
- Les payloads JSON restent minuscules (`{"image_id": "a1b2c3d4"}` = 30 octets)
- L'image est servie en binaire natif via `GET /api/image/{id}` (pas d'overhead d'encodage)
- On peut référencer la même image dans plusieurs requêtes sans la retransmettre

CORS

Le frontend tourne sur `localhost:5173`, le backend sur `localhost:8000`. Ce sont deux **origines différentes** (le port fait partie de l'origine). Par défaut, un navigateur interdit les requêtes cross-origin (politique same-origin, sécurité contre les attaques CSRF).

Le middleware CORS dans FastAPI ajoute les headers `Access-Control-Allow-Origin: http://localhost:5173` qui disent au navigateur « ce frontend a le droit de m'appeler ». En production (Docker), le frontend passe par nginx qui proxy les requêtes `/api/` vers le backend, donc on est sur la même origine et CORS n'est plus nécessaire.

---

3. Module prétraitement (TP2)

Le prétraitement prépare l'image avant la segmentation. C'est comme nettoyer et ajuster la luminosité d'une photo avant de la recadrer : si l'image est bruitée ou mal contrastée, toutes les méthodes de segmentation en aval souffriront.

3.1 Filtre gaussien

Principe mathématique : Chaque pixel est remplacé par une moyenne pondérée de ses voisins, où les poids suivent une courbe en cloche (distribution normale/ gaussienne) :

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2 + y^2}{2\sigma^2}}$$

Concrètement : plus un voisin est loin, moins il pèse dans la moyenne. Le paramètre σ (sigma) contrôle la largeur de la cloche : sigma grand = flou fort, sigma petit = flou léger.

Quand l'utiliser : Bruit gaussien (bruit de capteur blanc, aléatoire, additif). Typique des images IRM.

Paramètres : `sigma` (écart-type). La taille du noyau est calculée automatiquement : $k = 6\sigma + 1$ (arrondi à l'impair supérieur), ce qui couvre 99.7% de la distribution.

Limites : Le filtre gaussien **floute les contours** en même temps que le bruit. Sur une image d'endoscopie où les bords du polype sont déjà flous, c'est destructeur. C'est pourquoi on préfère le bilatéral.

Présence dans la plateforme : Filtre de base pour l'IRM, où le bruit est gaussien et les structures anatomiques sont larges.

3.2 Filtre médian

Principe : Chaque pixel est remplacé par la **médiane** (la valeur du milieu quand on les trie) de ses voisins dans un carré de taille $k \times k$.

Concrètement : si les voisins sont [10, 12, 200, 11, 13, 14, 12, 11, 15], la médiane est 13. Le pixel aberrant 200 est complètement ignoré.

Quand l'utiliser : **Bruit sel-et-poivre** (pixels isolés très noirs ou très blancs). Ce type de bruit apparaît lors de la transmission numérique d'images ou avec des capteurs défectueux.

Paramètre : `ksize` (taille du carré, doit être impair : 3, 5, 7...). Plus grand = plus de lissage mais perte de détails.

Limites : Mauvais pour le bruit gaussien (la médiane ne lisse pas bien le bruit continu). Peut créer des effets d'escalier sur les bords diagonaux.

Présence : Nettoyage initial sur les images bruitées avant segmentation.

3.3 Filtre moyenneur (mean)

Principe : Chaque pixel est remplacé par la **moyenne arithmétique** de ses voisins dans un disque de rayon $r = k/2$. C'est le filtre le plus simple : tous les voisins pèsent autant.

Quand l'utiliser : Lissage rapide et uniforme. Utile en première approximation.

Paramètre : `ksize` (diamètre du disque).

Limites : Floute les contours encore plus que le gaussien (pas de pondération décroissante). Sensible aux valeurs aberrantes (un seul pixel très clair tire la moyenne vers le haut).

Présence : Référence pédagogique du TP2, pour comparer avec le médian et le gaussien.

3.4 Filtre bilatéral

Principe : C'est un gaussien amélioré. Au lieu de ne pondérer que par la **distance spatiale**, il pondère aussi par la **similarité d'intensité** :

$$I'(x) = \frac{1}{W} \sum_{y \in \mathcal{N}(x)} I(y) \cdot \underbrace{e^{-\frac{|x-y|^2}{2\sigma_s^2}}}_{\text{poids spatial}} \cdot \underbrace{e^{-\frac{(I(x)-I(y))^2}{2\sigma_c^2}}}_{\text{poids d'intensité}}$$

Concrètement : un voisin qui a une intensité très différente (= il est de l'autre côté d'un contour) pèse presque zéro. Résultat : le bruit est lissé mais **les contours sont préservés**.

Pourquoi il préserve les contours (contrairement au gaussien) : Le gaussien ne regarde que la distance : un pixel à 2 pixels du contour sera moyenné avec des pixels de l'autre côté. Le bilatéral regarde aussi l'intensité : si un pixel est sombre et que son voisin est clair (contour), le poids d'intensité sera quasi-nul et le contour ne sera pas brouillé.

Paramètres :
- `d` : taille du voisinage (diamètre)
- `sigma_color` (σ_c) : tolérance en intensité. Grand = accepte plus de différence = flou plus fort
- `sigma_space` (σ_s) : portée spatiale du filtre

Limites : Lent (complexité $O(d^2)$ par pixel, pas séparable comme le gaussien). Peut créer un effet de postérisation (aplatissement des gradients doux) si sigma_color est trop petit.

Présence : **Critique pour l'endoscopie** : les images contiennent des polypes aux bords subtils et des reflets spéculaires. Le bilatéral lisse le bruit des tissus sans effacer les bords du polype.

3.5 Non-Local Means (NLM)

Principe : Au lieu de comparer un pixel avec ses voisins immédiats, NLM compare des **patchs** (petites fenêtres carrées) dans une zone de recherche étendue. Deux patchs similaires contribuent fortement, même s'ils sont spatialement éloignés.

$$I'(x) = \frac{1}{W} \sum_{y \in \text{search}} I(y) \cdot e^{-\frac{|P(x) - P(y)|^2}{h^2}} $$

où $P(x)$ est le patch centré en x et h contrôle le seuil de similarité.

Concrètement : si une texture (par exemple un vaisseau sanguin) se répète à plusieurs endroits de l'image, NLM les utilise toutes pour débruiter, ce qui donne un résultat bien meilleur que les filtres locaux.

Paramètres :
- `h` : force du débruitage. Trop grand = flou ; trop petit = le bruit reste
- `template_window` : taille du patch comparé (7×7 par défaut)
- `search_window` : taille de la zone de recherche (21×21 par défaut)

Limites : Très lent ($O(n \times s^2 \times t^2)$ par pixel). Pas adapté au temps réel.

Présence : Débruitage avancé pour les cas où le médian ou le gaussien ne suffisent pas.

3.6 CLAHE (Contrast Limited Adaptive Histogram Equalization)

Principe : L'image est découpée en **tuiles** (par exemple 8×8). Sur chaque tuile, on calcule l'histogramme et on l'égalise (redistribue les intensités pour occuper toute la plage [0, 255]). Mais avant d'égaliser, on **coupe** (clip) l'histogramme à un seuil maximum et on redistribue l'excédent uniformément. Enfin, les bordures des tuiles sont interpolées pour éviter les artefacts de blocs.

Pourquoi CLAHE est critique sur l'endoscopie : Les images d'endoscopie sont souvent globalement sombres (l'intérieur du corps est peu éclairé), avec quelques reflets spéculaires très brillants. L'égalisation d'histogramme standard (globale) amplifierait les reflets et créerait un contraste artificiel. CLAHE, en travaillant localement et en limitant l'amplification (clip), rehausse le contraste des zones sombres sans exploser les reflets.

Paramètres :
- `clip_limit` : seuil de coupure de l'histogramme. Plus élevé = plus de contraste

(mais risque de bruit amplifié). Défaut 2.0.
- `tile_grid_size` : nombre de tuiles. Plus petit = adaptation plus locale. Défaut 8×8.

Limites : Peut amplifier le bruit dans les zones très uniformes (parois de l'estomac par exemple).

3.7 Égalisation d'histogramme standard

Principe : On calcule la fonction de répartition cumulative (CDF) de l'histogramme et on l'utilise comme transformation d'intensité. L'objectif est d'obtenir un histogramme plat (uniforme) :

$$I'(x) = \text{CDF}(I(x)) \times 255$$

Concrètement : les intensités les plus fréquentes sont étalées, les moins fréquentes sont compressées.

Quand l'utiliser : Images dont l'histogramme est concentré dans une bande étroite (faible contraste global).

Limites : Traitement global → n'adapte pas le contraste localement. Sur une image d'endoscopie, elle peut amplifier les reflets et rendre l'image inexploitable. C'est exactement pour ça que CLAHE a été inventé.

3.8 Normalisation Z-score

Principe : Centrage-réduction statistique de chaque pixel :

$$I'(x) = \frac{I(x) - \mu}{\sigma}$$

puis rescaling dans [0, 255].

Concrètement : après Z-score, la moyenne de l'image est 0 et l'écart-type est 1. Cela normalise les intensités entre images acquises avec des protocoles différents.

Quand l'utiliser : Quand on compare ou traite des images acquises à des moments différents ou sur des scanners différents. Étape de prétraitement standard avant deep learning.

Limites : Si l'image est quasi-constante ($\sigma \approx 0$), la division diverge. Le code gère ce cas en retournant l'image inchangée.

3.9 Correction N4 bias field

Principe : En IRM, le champ magnétique n'est jamais parfaitement homogène. Cela crée un **biais de champ** : une variation lente et douce des intensités sur l'image (un côté est plus clair que l'autre, par exemple). N4 modélise ce biais par un polynôme multiplicatif et le corrige itérativement :

$$I_{\text{observée}}(x) = I_{\text{vraie}}(x) \times B(x) + \text{bruit}$$

L'algorithme estime $B(x)$ (le champ de biais) par une B-spline et divise l'image par ce champ.

Pourquoi N4 est spécifique à l'IRM : Le biais de champ est un artefact physique de l'IRM (inhomogénéités des bobines RF). Il n'existe pas en endoscopie (qui utilise une caméra optique) ni en scanner CT. L'appliquer sur une image non-IRM ne ferait que déformer les intensités.

Paramètres : Nombre d'itérations (50 par niveau), 4 niveaux de résolution.

Limites : Lent (plusieurs secondes par image). Nécessite un masque de l'objet (calculé automatiquement par seuillage Otsu). Ne corrige pas le bruit — seulement la variation lente.

---

4. Module segmentation classique (TP2 + TP3)

La segmentation est le cœur du projet : séparer automatiquement la zone d'intérêt (polype, tumeur) du fond. C'est comme découper un objet dans une photo, mais avec des contraintes médicales : les bords sont flous, le contraste est faible, et une erreur peut avoir des conséquences cliniques.

4.1 Seuillage d'Otsu

Principe : On cherche un seuil T qui sépare l'image en deux classes (fond et objet) en maximisant la **variance inter-classes** :

$$\sigma^2_B(T) = \omega_0(T) \cdot \omega_1(T) \cdot [\mu_0(T) - \mu_1(T)]^2$$

où ω_0, ω_1 sont les proportions de pixels de chaque classe et μ_0, μ_1 leurs moyennes.

Concrètement : Otsu teste tous les seuils possibles (0 à 255) et garde celui qui maximise la séparation entre les deux groupes de pixels. C'est comme trouver le meilleur endroit pour couper un histogramme bimodal en deux.

Algorithme :

```
Pour T = 0 à 255 :
    Classe 0 = pixels ≤ T
    Classe 1 = pixels > T
    Calculer variance inter-classes
Retourner T qui maximise la variance
Masque = (image > T) × 255
```

Cas d'usage : Fonctionne bien quand l'histogramme est **bimodal** (deux pics distincts : un pour le fond, un pour l'objet). Exemple : IRM cérébrale avec tumeur contrastée.

Résultat sur endoscopie vs IRM :
- **IRM** : Souvent correct si le contraste tumeur/fond est bon (après N4 + CLAHE)

- **Endoscopie** : Souvent médiocre — les polypes ont des intensités proches du tissu environnant, l'histogramme n'est pas bimodal

Limites : Échoue si l'histogramme est unimodal (pas de séparation naturelle), si le bruit est fort, ou si l'éclairage est hétérogène. Produit un seuil **global** : même seuil partout sur l'image.

4.2 Seuillage adaptatif

Principe : Au lieu d'un seuil global, on calcule un seuil **local** pour chaque pixel, basé sur la moyenne pondérée (gaussienne) de ses voisins dans un bloc de taille $B \times B$:

$$T(x, y) = \text{moyenne_pondérée}(\text{voisinage de } B \times B) - C$$

Le pixel est classé blanc si $I(x, y) > T(x, y)$, noir sinon.

Concrètement : le seuil s'adapte aux conditions locales de luminosité. Dans une zone sombre, le seuil sera bas ; dans une zone claire, il sera haut.

Paramètres :
- `block_size` (B) : taille du voisinage. Doit être impair et > 1. Grand = lissage du seuil.
- `C` : constante soustraite. Permet d'ajuster la sensibilité.

Cas d'usage : Images avec éclairage hétérogène. Beaucoup mieux qu'Otsu quand la luminosité varie (endoscopie avec lumière directionnelle).

Limites : Sensible au choix de B et C. Peut créer du bruit de seuillage (pixels isolés mal classés).

4.3 Watershed (Ligne de partage des eaux)

Principe : On imagine l'image comme un relief topographique (les intensités sont des altitudes). On « inonde » le relief depuis des sources (marqueurs). Quand l'eau de deux sources se rencontre, on trace un barrage = la frontière de segmentation.

Analogie : imaginez que vous percez des trous dans une carte en relief et que vous versez de l'eau par le dessous. Chaque trou est un marqueur. Quand deux bassins d'eau se touchent, la ligne de contact est la frontière.

Pourquoi le watershed sur-segmente sans marqueurs : Sans marqueurs, chaque minimum local de l'image (chaque petit creux) crée sa propre source. Sur une image médicale bruitée, il y a des centaines de minima locaux → des centaines de régions minuscules. C'est la **sur-segmentation**.

Comment on construit les marqueurs dans notre code :

```
1. Otsu → masque binaire grossier
2. Ouverture morphologique → supprime le bruit
3. Dilatation → zone "sure fond" (sure_bg)
4. Distance transform → pour chaque pixel blanc, sa distance au noir le plus proche
5. Seuillage à 50% du max → zone "sure premier plan" (sure_fg)
6. sure_bg - sure_fg → zone "incertaine" (unknown)
7. connectedComponents sur sure_fg → un marqueur par composante connexe
8. Marqueurs[unknown] = 0 → "à déterminer"
9. cv2.watershed(image_couleur, marqueurs) → segmentation finale
```

Résultat attendu :
- **IRM** : Bon pour séparer les lobes ou structures bien contrastées
- **Endoscopie** : Résultat variable, dépend beaucoup de la qualité des marqueurs

Limites : Sensible au bruit (bruit = faux minima = sur-segmentation). Nécessite un bon prétraitement et des marqueurs bien choisis.

4.4 Chan-Vese (contours actifs sans bord)

Principe : Méthode par **level-set** qui cherche un contour C qui minimise l'énergie :

$$E = \mu \cdot \text{Longueur}(C) + \lambda_1 \int_{\text{intérieur}} (I - c_1)^2 + \lambda_2 \int_{\text{extérieur}} (I - c_2)^2$$

où c_1 et c_2 sont les moyennes d'intensité à l'intérieur et à l'extérieur du contour.

Concrètement : le contour évolue itérativement pour englober les pixels dont l'intensité est proche de c_1 (la moyenne intérieure) et exclure ceux proches de c_2 (la moyenne extérieure). Le terme μ empêche le contour de devenir trop irrégulier.

Paramètres :
- `mu` : poids de régularisation (longueur du contour). Grand = contour lisse. Petit = contour peut épouser les détails.
- `lambda1`, `lambda2` : poids des termes d'attache aux données intérieur/extérieur
- `max_iter` : nombre d'itérations

Cas d'usage : Images où les bords ne sont pas marqués par un gradient fort (contours « sans bord » — d'où le nom). Typiquement : images médicales avec des transitions douces.

Résultat sur endoscopie : Souvent meilleur qu'Otsu car il ne dépend pas d'un histogramme bimodal. Mais lent (200 itérations par défaut).

Limites : Converge vers un minimum local (le résultat dépend de l'initialisation). Lent. Ne gère pas naturellement les objets multiples.

4.5 K-means

Principe : On traite chaque pixel comme un point dans un espace de caractéristiques (ici : juste l'intensité) et on les regroupe en k clusters par l'algorithme K-means :

```
1. Initialiser k centres aléatoirement
2. Répéter :
   a. Assigner chaque pixel au centre le plus proche
   b. Recalculer les centres (moyenne de chaque cluster)
   Jusqu'à convergence
3. Le cluster le plus brillant = foreground
```

Concrètement : K-means avec k=2 sépare l'image en « sombre » et « clair ». Avec k=3, on ajoute une classe intermédiaire.

Paramètre : `k` (nombre de clusters, défaut 3).

Limites :
- Résultat dépend de l'initialisation aléatoire (on lance 10 essais et on garde le meilleur)
- Suppose des clusters sphériques dans l'espace des intensités (pas toujours vrai)
- Ne prend pas en compte la position spatiale (deux régions identiques en intensité mais éloignées sont fusionnées)

4.6 Mean Shift

Principe : Algorithme de clustering non-paramétrique. Chaque pixel se déplace vers la **mode** (le pic de densité) locale de la distribution des intensités/couleurs voisines. Les pixels qui convergent vers le même mode forment un cluster.

Concrètement : imaginez que chaque pixel est une boule qui roule vers le sommet de la colline la plus proche. Les boules qui arrivent au même sommet sont dans le même cluster.

Dans notre implémentation, on utilise `cv2.pyrMeanShiftFiltering` qui filtre l'image par Mean Shift (chaque pixel prend la couleur du mode local), puis on applique Otsu sur le résultat filtré.

Paramètres :
- `spatial_radius` : distance spatiale de recherche
- `color_radius` : tolérance en intensité/couleur

Avantage : Pas besoin de spécifier k (le nombre de clusters est trouvé automatiquement).

Limites : Lent ($O(n^2)$ naïf). Le résultat dépend des rayons choisis.

4.7 Croissance de régions (Region Growing)

Principe : On part d'un **point graine** (seed) choisi par l'utilisateur et on fait croître la région par propagation BFS (parcours en largeur) :

```
1. File = [(seed_y, seed_x)]
2. Tant que File non vide :
   a. Dépiler un pixel (y, x)
   b. Si |intensité(y,x) - intensité(seed)| ≤ seuil :
      - Ajouter au masque
      - Empiler les 4 voisins non visités
```

Concrètement : la région grandit comme une tache d'eau qui se propage aux pixels voisins tant qu'ils sont « assez similaires » au pixel de départ.

Paramètres :
- `seed_x`, `seed_y` : position du point de départ
- `threshold` : écart d'intensité maximum toléré

Cas d'usage : Quand on sait où se trouve l'objet et qu'on veut délimiter précisément ses bords. Utile en mode semi-automatique (le clinicien clique sur la lésion).

Limites : Totalement dépendant du choix du seed. Peut fuir (leak) si le seuil est trop grand ou si un contour a un trou.

---

5. Module deep learning (TP4 + améliorations)

5.1 U-Net : architecture détaillée

Pourquoi un U-Net plutôt qu'un simple CNN de classification

Un CNN de classification (comme ResNet, VGG) prend une image et produit **un label** (« tumeur » ou « pas tumeur »). Un U-Net produit **un masque** de la même taille que l'image d'entrée, où chaque pixel est classé individuellement. C'est la différence entre dire « il y a une tumeur dans cette image » et « la tumeur occupe exactement ces pixels ».

Pour produire un masque pixel par pixel, il faut à la fois :
- Comprendre le **contexte global** (est-ce une tumeur ou un artefact ?) → vision large
- Localiser précisément les **contours** → vision fine

Le U-Net résout ce problème avec sa forme en U : l'encodeur réduit progressivement la résolution pour capter le contexte, le décodeur la restaure pour localiser, et les **skip connections** transmettent les détails fins de l'encodeur au décodeur.

Encodeur (chemin contractant)

```
Entrée : 256 × 256 × 1 (image en niveaux de gris)

Bloc 1 : 2 × Conv2D(16, 3×3, ReLU, he_normal) → Dropout(0.1) → MaxPool(2×2)
          → 128 × 128 × 16

Bloc 2 : 2 × Conv2D(32, 3×3, ReLU, he_normal) → Dropout(0.1) → MaxPool(2×2)
          → 64 × 64 × 32

Bloc 3 : 2 × Conv2D(64, 3×3, ReLU, he_normal) → Dropout(0.2) → MaxPool(2×2)
          → 32 × 32 × 64

Bloc 4 : 2 × Conv2D(128, 3×3, ReLU, he_normal) → Dropout(0.2) → MaxPool(2×2)
          → 16 × 16 × 128

Bottleneck : 2 × Conv2D(256, 3×3, ReLU, he_normal) → Dropout(0.3)
          → 16 × 16 × 256
```

Pourquoi doubler les filtres : À chaque pooling, la résolution spatiale est divisée par 2 (on perd des détails spatiaux). Pour compenser, on double le nombre de filtres (16→32→64...) : on perd en « où » mais on gagne en « quoi ». Chaque niveau capture des caractéristiques de plus en plus abstraites.

he_normal : Initialisation de He, conçue pour les activations ReLU. Elle initialise les poids avec une variance de $2/n_{in}$ (où n_{in} est le nombre d'entrées). Cela évite le problème de vanishing/exploding gradients.

MaxPooling(2×2) : Prend le maximum de chaque carré 2×2. Réduit la taille par 2 et rend le réseau localement invariant aux petites translations.

Décodeur (chemin expansif)

```
Bloc 6 : Conv2DTranspose(128, 2×2, stride 2) → Concaténation avec Bloc 4
          → 2 × Conv2D(128) → Dropout(0.2)
          → 32 × 32 × 128

Bloc 7 : Conv2DTranspose(64, 2×2, stride 2) → Concaténation avec Bloc 3
          → 2 × Conv2D(64) → Dropout(0.2)
          → 64 × 64 × 64

Bloc 8 : Conv2DTranspose(32, 2×2, stride 2) → Concaténation avec Bloc 2
          → 2 × Conv2D(32) → Dropout(0.1)
          → 128 × 128 × 32

Bloc 9 : Conv2DTranspose(16, 2×2, stride 2) → Concaténation avec Bloc 1
          → 2 × Conv2D(16) → Dropout(0.1)
          → 256 × 256 × 16

Sortie : Conv2D(1, 1×1, sigmoid) → 256 × 256 × 1
```

Conv2DTranspose (déconvolution ou convolution transposée) : c'est l'inverse du pooling — elle double la résolution spatiale. On peut la voir comme un upsampling appris (pas juste une interpolation bilinéaire).

Skip connections

Les skip connections relient chaque bloc de l'encodeur au bloc symétrique du décodeur par **concaténation**. Sans elles, le décodeur devrait reconstruire les détails fins à partir du bottleneck 16×16, ce qui est presque impossible.

Avec les skip connections, le décodeur reçoit :
- Du bottleneck : « il y a un objet au centre de l'image » (contexte)
- Du skip : « voici les contours détaillés à cette résolution » (détails)

C'est comme donner à un dessinateur à la fois le plan global ET les croquis de détails.

Sigmoid en sortie

La sortie est un seul canal avec activation sigmoid : $\sigma(x) = 1/(1 + e^{-x})$. Chaque pixel a une valeur entre 0 et 1, interprétée comme la **probabilité d'appartenir au premier plan**. On seuille à 0.5 pour obtenir le masque binaire.

5.2 Entraînement

Pourquoi la BCE seule échoue sur l'endoscopie

La Binary Cross-Entropy (BCE) pénalise chaque pixel indépendamment :

$$\text{BCE} = -\frac{1}{N}\sum_{i=1}^{N} [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$$

Sur le dataset d'endoscopie, les polypes occupent souvent 5-10% de l'image. Le fond = 90-95%. Si le modèle prédit « tout est fond » (masque noir), il a déjà 90-95% de précision pixel par pixel. La BCE le récompense pour cette solution paresseuse, d'où le Dice = 0 observé au TP4 à 10 epochs.

La perte combinée Dice + BCE

La Dice loss mesure directement le chevauchement entre la prédiction et la vérité terrain :

$$\text{Dice Loss} = 1 - \frac{2 \sum_i y_i \hat{y}_i + \epsilon}{\sum_i y_i + \sum_i \hat{y}_i + \epsilon}$$

Elle pénalise le modèle s'il ne segmente pas le foreground, même s'il est petit. Combinée à la BCE :

$$\mathcal{L} = 0.5 \times \text{BCE} + 0.5 \times \text{Dice Loss}$$

La BCE apporte la stabilité du gradient (dérivée bien définie partout), la Dice loss force le modèle à se soucier du foreground minoritaire.

Data augmentation

Le dataset d'endoscopie ne contient que 100 images. Sans augmentation, le modèle mémoriserait les 100 images (overfitting). L'augmentation crée des variations artificielles :

- **HorizontalFlip / VerticalFlip** : un polype à gauche ou à droite est toujours un polype
- **Rotate(±15°)** : l'endoscope ne cadre pas toujours droit
- **RandomBrightnessContrast** : simule des conditions d'éclairage différentes
- **ElasticTransform** : **pertinent en médical** car les tissus biologiques se déforment naturellement. Simule les mouvements de respiration, de péristaltisme. Cette augmentation est plus réaliste que du simple cisaillement pour les images d'organes mous.

La synchronisation image/masque est cruciale : quand on flippe l'image, on flippe le masque identiquement.

ModelCheckpoint sur val_dice (pas val_loss)

La loss (BCE + Dice) mélange deux termes. Un modèle peut avoir une loss plus basse tout en ayant un Dice plus mauvais (s'il améliore la BCE sur le fond au détriment du Dice sur le foreground). En surveillant directement le `val_dice_metric`, on sauvegarde le modèle qui segmente le mieux, pas celui qui minimise le mieux un proxy.

EarlyStopping patience 15

Si le Dice de validation n'augmente plus pendant 15 epochs consécutives, on arrête l'entraînement. Pourquoi 15 et pas 5 ? Parce que l'entraînement sur un petit dataset est bruité : le Dice peut stagner pendant 5-10 epochs puis remonter grâce au ReduceLROnPlateau qui divise le learning rate. 15 laisse assez de marge pour ces plateaux.

5.3 Évolution par rapport au TP4

Problème du TP4	Cause	Correction apportée
Dice = 0 à 10 epochs	BCE seule → le modèle prédit tout fond	Perte combinée Dice + BCE qui force la détection du foreground
Dice = 0.47 à 50 epochs	Overfitting sur 100 images	Augmentation (flip, rotation, elastic) + Dropout
Pas de sauvegarde du meilleur modèle	Le modèle final est celui de la dernière epoch (pas forcément le meilleur)	ModelCheckpoint sur val_dice_metric
Entraînement à durée fixe	Peut s'arrêter trop tôt ou trop tard	EarlyStopping patience=15
Learning rate fixe	Peut osciller autour du minimum	ReduceLROnPlateau (÷2 si plateau de 5 epochs)

---

6. Extension recherche : Monte Carlo Dropout

6.1 Problème

Un U-Net standard produit un masque binaire : chaque pixel est classé 0 (fond) ou 1 (objet). Il n'y a aucune **mesure de confiance**. Le pixel est à 0.51 (« probablement objet, mais je ne suis pas sûr ») ou à 0.99 (« quasi-certain ») ? Impossible à savoir.

Dans un contexte clinique, c'est problématique. Un radiologue qui utilise un outil de segmentation automatique doit savoir **quand** faire confiance au modèle et **quand** vérifier manuellement.

Incertitude épistémique vs aléatoire

- **Incertitude épistémique** (du grec *episteme* = connaissance) : le modèle ne sait pas parce qu'**il n'a pas assez appris**. Avec plus de données d'entraînement, cette incertitude diminuerait. Exemple : une tumeur d'un type rare jamais vu en entraînement.
- **Incertitude aléatoire** (du latin *alea* = hasard) : l'incertitude est **inhérente aux données** et ne peut pas être réduite, même avec un modèle parfait. Exemple : le bruit de capteur IRM brouille le contour réel de la tumeur.

MC Dropout capture l'incertitude épistémique : la variance entre les passes reflète la sensibilité du modèle à ses propres poids, pas au bruit des données.

6.2 Principe du MC Dropout (Gal & Ghahramani, 2016)

Normalement

Pendant l'entraînement, Dropout désactive aléatoirement une fraction p des neurones à chaque forward pass. Cela force le réseau à ne pas dépendre d'un seul neurone → régularisation → moins d'overfitting.

À l'inférence (prédiction), Dropout est **désactivé** : tous les neurones sont actifs, et les poids sont multipliés par $(1-p)$ pour compenser. On obtient une prédiction unique et déterministe.

Avec MC Dropout

On **garde Dropout actif** à l'inférence. Chaque forward pass tire un sous-ensemble aléatoire de neurones, ce qui donne un **sous-réseau différent** à chaque fois. On répète N fois :

```
Pour t = 1 à N :
    Activer Dropout (masque aléatoire différent)
    pred_t = model(image, training=True)  # training=True garde le Dropout

Prédiction moyenne : p̂ = (1/N) Σ pred_t
Incertitude (std)  : σ = std(pred_1, ..., pred_N)
```

Mathématiquement : chaque forward pass avec Dropout échantillonne un modèle $f_{\hat{\theta}_t}$ de l'espace a posteriori des poids. La moyenne des prédictions approxime l'espérance bayésienne $\mathbb{E}[f(\theta) | \text{données}]$ et la variance approxime l'incertitude épistémique.

Plus rigoureusement, Gal & Ghahramani (2016) montrent que l'entraînement avec Dropout est équivalent à une **approximation variationnelle** d'un **processus gaussien** (GP). Chaque masque de Dropout définit une distribution $q(\theta)$ qui approxime la vraie distribution a posteriori $p(\theta | \text{données})$. Les N passes de MC Dropout échantillonnent cette approximation.

Intuition : Si tous les sous-réseaux sont d'accord (faible variance), le modèle est confiant. S'ils divergent (forte variance), le modèle hésite — probablement parce qu'il n'a pas vu assez d'exemples similaires en entraînement.

6.3 Implémentation concrète

Le `training=True` dans Keras

```
@tf.function
def _stochastic_forward(model, x):
    return model(x, training=True)
```

En Keras, le paramètre `training` contrôle le comportement des couches qui agissent différemment en train vs inférence :
- **Dropout** : actif si `training=True`, désactivé sinon
- **BatchNormalization** : utilise les stats du batch si `training=True`, les stats mobiles sinon

Avec `training=True`, Dropout reste actif et chaque appel produit un résultat différent. Le `@tf.function` compile le graphe TensorFlow pour accélérer l'exécution.

Notre U-Net n'utilise pas de BatchNorm (conformément au TP4), donc `training=True` n'affecte que le Dropout — exactement ce qu'on veut.

Calcul des sorties

N prédictions : on empile les N résultats (chacun est une matrice 256×256 de probabilités [0, 1]) :

```
preds = np.stack([_stochastic_forward(model, x).numpy()[0, ..., 0]
                  for _ in range(n_passes)], axis=0)
# preds.shape = (N, 256, 256)
```

Prédiction moyenne :

```
mean_prob = preds.mean(axis=0)  # (256, 256), valeurs dans [0, 1]
```

Carte d'écart-type (incertitude pixel par pixel) :

```
std_map = preds.std(axis=0)  # (256, 256), valeurs ≥ 0
```

Un pixel à std=0.0 : tous les sous-réseaux sont d'accord → confiant.
Un pixel à std=0.3 : les sous-réseaux divergent → incertain.

Carte d'entropie prédictive :

$$H(p) = -[p \cdot \log(p) + (1-p) \cdot \log(1-p)]$$

L'entropie mesure l'incertitude d'une distribution de Bernoulli. Elle est maximale à $p = 0.5$ (incertitude maximale : pile ou face) et vaut 0 à $p = 0$ ou $p = 1$ (certitude).

```
p = np.clip(mean_prob, 1e-7, 1 - 1e-7)  # éviter log(0)
entropy_map = -(p * np.log(p) + (1 - p) * np.log(1 - p))
```

Score de confiance global :

$$\text{Confiance} = 1 - \frac{\bar{H}_{\text{foreground}}}{\log(2)}$$

On normalise par $\log(2)$ (entropie maximale d'une Bernoulli) pour obtenir un score dans [0, 1]. On ne calcule la moyenne que sur le foreground (les pixels prédits comme objet) car c'est la zone cliniquement pertinente.

Code couleur du score :
- Vert (≥ 0.8) : confiant
- Orange (0.6 - 0.8) : modérément confiant
- Rouge (< 0.6) : incertain → le clinicien doit vérifier

6.4 Mini-étude benchmarkée (notebook)

> *Note : les valeurs numériques exactes dépendent du modèle entraîné. Le notebook `uncertainty_study.ipynb` les calculera sur tes données. Voici la structure et l'interprétation attendue.*

Expérience A — Sensibilité à N

On fait varier $N \in {5, 10, 20, 30, 50}$ et on mesure :
- **Dice moyen** : le masque de prédiction moyenne est-il meilleur avec plus de passes ? En théorie, oui (loi des grands nombres), mais le gain sature.
- **Confiance moyenne** : se stabilise-t-elle ? Oui, la variance de l'estimateur diminue en $1/\sqrt{N}$.
- **Temps d'inférence** : proportionnel à N. Sur CPU, chaque passe prend ~0.3s, donc N=30 → ~10s.

Résultat attendu : le Dice et la confiance se stabilisent vers N=20-30 (plateau). Au-delà, le gain marginal ne justifie plus le coût en temps. On retient **N=30** comme compromis.

Expérience B — Corrélation incertitude ↔ erreur

Pour chaque image i, on calcule :
- $\bar{\sigma}_i$ = incertitude moyenne (mean de la carte std)
- $e_i = 1 - \text{Dice}_i$ = erreur de segmentation

On trace le scatter plot $(\bar{\sigma}_i, e_i)$ et on calcule la corrélation de Pearson :

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

Interprétation :
- $r > 0.5$ avec $p < 0.05$: **le modèle sait quand il ne sait pas**. C'est le résultat idéal : les cas où il échoue sont ceux où il signale une forte incertitude.
- $r \leq 0.5$ ou $p \geq 0.05$: la corrélation est faible ou non significative. Le modèle est mal calibré — l'incertitude ne prédit pas l'erreur.

Expérience C — Rejet sélectif

On trie les images par confiance décroissante et on rejette les τ% les moins confiantes :

τ (%)	Images retenues	Dice moyen
0	10/10	Dice_all
20	8/10	Dice_20
40	6/10	Dice_40

Message clé (pour le CV et le rapport) : « Rejeter les 20% des prédictions les moins confiantes fait passer le Dice de X à Y (+Z%). »

C'est un scénario de **triage clinique** : le radiologue fait confiance au modèle sur 80% des cas et vérifie manuellement les 20% signalés comme incertains.

Expérience D — Galerie de 3 cas

Cas	Confiance	Dice	Signification
1	Haute	Bon	Le modèle fonctionne bien et il le sait → OK
2	Haute	Mauvais	**Danger** : le modèle se trompe mais reste confiant → faux sentiment de sécurité. Souvent causé par un cas hors distribution (out-of-distribution)
3	Basse	Mauvais	Le modèle échoue et il l'admet → **honnête**. C'est le cas idéal de MC Dropout : le clinicien est averti

Le cas 2 est le plus intéressant en discussion : il montre les limites du MC Dropout (il ne peut pas tout détecter).

---

7. Module évaluation (TP3 + TP5)

7.1 Dice (coefficient de Sørensen-Dice)

Formule :

$$\text{Dice} = \frac{2 |A \cap B|}{|A| + |B|}$$

où A = vérité terrain, B = prédiction, $|\cdot|$ = nombre de pixels.

Plage : [0, 1]. 0 = aucun chevauchement, 1 = chevauchement parfait.

Interprétation concrète : c'est le ratio de pixels bien classés *dans l'objet* rapporté au nombre total de pixels de l'objet (dans les deux masques). Un Dice de 0.8 signifie que 80% de l'objet est correctement couvert.

Quand il ment : Si l'objet est minuscule (5 pixels) et que la prédiction en couvre 4, Dice = 2×4/(5+4) ≈ 0.89. Ça semble bon, mais on a raté 20% de l'objet. Sur des petites structures, même une erreur de 1 pixel change beaucoup le Dice.

7.2 IoU (Intersection over Union, ou Jaccard)

Formule :

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|}$$

Plage : [0, 1]. Même interprétation que Dice mais **toujours ≤ Dice** (pour le même résultat).

Différence Dice vs IoU : Pour un même résultat, le Dice est toujours plus élevé que l'IoU. La relation exacte est :

$$\text{Dice} = \frac{2 \times \text{IoU}}{1 + \text{IoU}}$$

IoU	Dice
0.50	0.67
0.75	0.86
0.90	0.95

Pourquoi on utilise les deux : La communauté de segmentation médicale rapporte traditionnellement le Dice (plus optimiste, convention médicale). La communauté de vision par ordinateur rapporte l'IoU (convention COCO, PASCAL VOC). On fournit les deux pour permettre la comparaison avec la littérature des deux domaines.

L'IoU pénalise plus sévèrement les petites erreurs : si on ajoute 10 faux positifs à une petite intersection, le dénominateur (union) augmente de 10 et l'IoU chute fortement. Le Dice est plus indulgent.

7.3 Corrélation de Pearson

Formule :

$$r = \frac{\sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2} \sqrt{\sum (y_i - \bar{y})^2}}$$

Plage : [-1, 1]. +1 = corrélation positive parfaite, 0 = pas de corrélation linéaire, -1 = corrélation négative parfaite.

Usage dans le projet :
- En TP5 : mesurer la similarité entre deux images (recalage)
- En expérience B : corrélation incertitude ↔ erreur

Quand il ment : Ne détecte que les relations **linéaires**. Deux images identiques à un décalage spatial près auront un Pearson < 1.

7.4 MSE (Mean Squared Error)

Formule :

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^{N} (I_1(i) - I_2(i))^2$$

Plage : [0, +∞). 0 = images identiques.

Usage : Mesure de dissimilarité. Plus MSE est petit, plus les images sont proches. Sensible aux outliers (un seul pixel très différent peut dominer le MSE).

7.5 NMI (Normalized Mutual Information)

Formule (convention TP5) :

$$\text{NMI} = \frac{H(X) + H(Y)}{H(X, Y)}$$

où $H(X) = -\sum p(x) \log p(x)$ est l'entropie marginale et $H(X, Y) = -\sum p(x,y) \log p(x,y)$ est l'entropie conjointe.

Plage : [1, +∞) avec cette convention. NMI = 1 si les images sont indépendantes. NMI augmente avec la dépendance.

Pourquoi NMI est utile en imagerie médicale : La NMI mesure la **quantité d'information partagée** entre deux images, indépendamment de la relation d'intensité. Cruciale pour le **recalage multimodal** (aligner une IRM T1 avec une IRM T2, ou une IRM avec un CT) : les intensités ne correspondent pas entre modalités (un os est blanc en CT mais noir en IRM T2), mais la NMI capture quand même que les structures se correspondent.

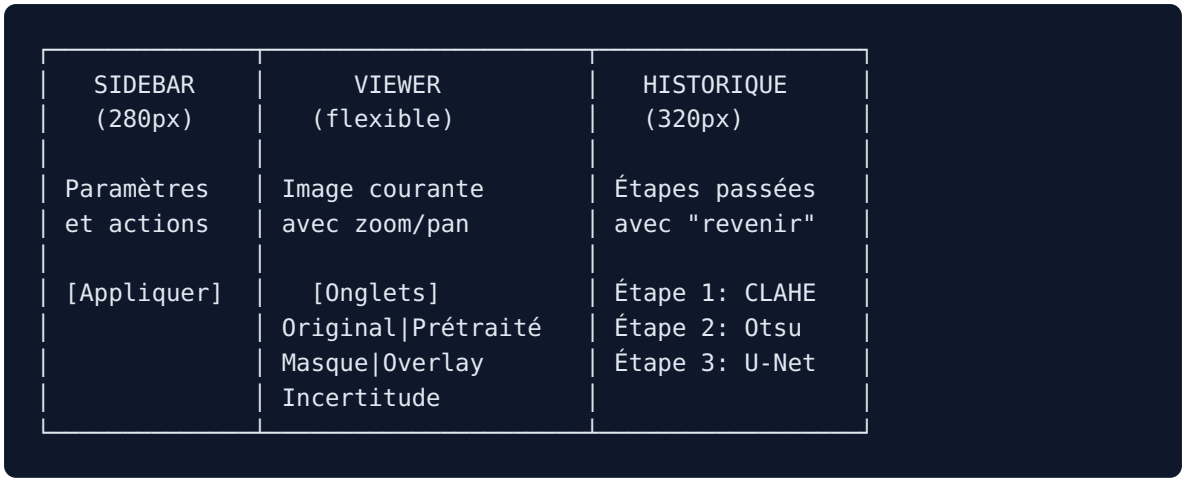
Quand elle ment : Sur des images très petites ou très uniformes, l'estimation de l'histogramme joint est bruitée (peu de bins occupés).

---

8. Interface utilisateur

Pourquoi 3 colonnes dans le Workspace

La disposition en 3 colonnes suit le flux de travail naturel :



- **Gauche** : contrôles et paramètres (l'utilisateur configure)
- **Centre** : résultat visuel (l'utilisateur observe)
- **Droite** : historique (l'utilisateur navigue dans le pipeline)

C'est un pattern classique d'outil professionnel (Photoshop, GIMP, 3D Slicer).

Code couleur des masques

- **Rouge** `rgba(220, 38, 38, 0.5)` : masque prédit (segmentation automatique)
- **Vert** `rgba(34, 197, 94, 0.5)` : masque ground truth (vérité terrain annotée manuellement)

Le rouge pour le prédit suit la convention médicale où le rouge signale une zone détectée automatiquement (à vérifier). Le vert pour la vérité terrain représente la référence « validée ». L'overlay semi-transparent permet de voir l'image sous-jacente.

Pourquoi un design sobre

Le design suit les principes des outils cliniques : pas de gradients, pas d'ombres, pas de glassmorphism. Raisons :
- **Neutralité visuelle** : les couleurs de l'interface ne doivent pas interférer avec l'interprétation des images médicales
- **Fatigue visuelle** : un radiologue travaille des heures devant l'écran, un design minimal réduit la fatigue
- **Crédibilité** : un outil qui ressemble à un template SaaS coloré ne sera pas pris au sérieux dans un contexte médical

Références : Linear (gestion de projet), Vercel Dashboard, 3D Slicer (imagerie).

Raccourcis clavier

Touche	Action	Motivation
`o`	Overlay	**O**verlay — comparaison rapide pred/image
`m`	Masque seul	**M**asque — voir la segmentation sans l'image
`r`	Original	**R**eset — retour à l'image de base
`u`	Incertitude	**U**ncertainty — voir la carte MC Dropout
`1-9`	Étape N du pipeline	Navigation rapide dans l'historique

---

9. Questions pièges d'entretien — anticipation

Q1 : « Pourquoi Keras et pas PyTorch ? »

Réponse : Le TP4 utilise Keras/TensorFlow. Garder la même stack assure la cohérence avec les travaux pratiques et permet de réutiliser directement le code d'entraînement. De plus, le MC Dropout est trivial à implémenter en Keras (un seul paramètre `training=True`), alors qu'en PyTorch il faudrait mettre `model.train()` ce qui active aussi le BatchNorm — gestion supplémentaire nécessaire. Sur le fond, les deux frameworks sont équivalents pour ce projet.

Q2 : « Votre Dice est modeste (~0.47 en TP4, objectif 0.75). Pourquoi ne pas avoir utilisé nnU-Net ? »

Réponse : L'objectif pédagogique est de **comprendre** l'architecture, pas d'obtenir le meilleur score. nnU-Net automatise le choix de l'architecture, des hyperparamètres et du prétraitement — c'est une boîte noire. Notre U-Net simplifié permet de comprendre chaque brique (skip connections, Dropout, Conv2DTranspose) et de justifier les choix. De plus, le dataset de 100 images est trop petit pour exploiter la complexité de nnU-Net. En perspective, on pourrait comparer avec nnU-Net ou SAM sur un dataset plus grand.

Q3 : « Comment MC Dropout diffère-t-il d'un ensemble de modèles ? »

Réponse : Un ensemble de modèles (deep ensemble) entraîne N modèles séparés avec des initialisations différentes. MC Dropout n'entraîne qu'**un seul** modèle et simule l'ensemble en variant les masques de Dropout. Avantage : N× moins d'entraînement, N× moins de mémoire pour stocker les poids. Inconvénient : la diversité des sous-réseaux est limitée (ils partagent les mêmes poids), donc l'incertitude est sous-estimée par rapport à un vrai ensemble. Dans la littérature, les deep ensembles donnent de meilleures incertitudes mais à un coût N× plus élevé.

Q4 : « Pourquoi 30 passes et pas 100 ? »

Réponse : L'expérience A du notebook montre que le Dice moyen et la confiance se stabilisent vers N=20-30 (plateau). Au-delà, le gain marginal est négligeable (la variance de l'estimateur diminue en $1/\sqrt{N}$, donc passer de 30 à 100 divise l'erreur par $\sqrt{100/30} \approx 1.8$ seulement). Sur CPU, chaque passe prend ~0.3s, donc N=100 → 30 secondes par image, ce qui est prohibitif en usage interactif.

Q5 : « Que se passe-t-il si la carte d'incertitude n'est PAS corrélée à l'erreur ? »

Réponse : Cela signifierait que le modèle est **mal calibré** : il est confiant quand il ne devrait pas l'être et incertain quand il a raison. Causes possibles : (1) le Dropout rate est trop faible pour créer assez de variabilité, (2) le modèle n'a pas assez de capacité pour que les sous-réseaux divergent, (3) le dataset est trop homogène (toutes les images se ressemblent). Solutions : augmenter le Dropout rate, utiliser un ensemble de modèles, ou entraîner avec une perte calibrée (temperature scaling).

Q6 : « Comment vous entraînez sur un dataset de 100 images sans overfit ? »

Réponse : Trois mécanismes principaux. (1) **Dropout** : désactive aléatoirement des neurones → le réseau ne peut pas mémoriser des patterns spécifiques à une image. (2) **Data augmentation** (flips, rotations, elastic transform) : multiplie artificiellement

la diversité du dataset. (3) **EarlyStopping** sur la validation : arrête l'entraînement quand le Dice de validation stagne, avant que l'overfitting ne commence. De plus, notre U-Net est relativement petit (~2M paramètres), ce qui limite sa capacité de mémorisation.

Q7 : « Qu'est-ce qui différencie votre watershed d'un watershed naïf ? »

Réponse : Un watershed naïf inonde depuis tous les minima locaux → sur-segmentation extrême (des centaines de petites régions). Notre implémentation construit des **marqueurs intelligents** via (1) Otsu pour une binarisation grossière, (2) ouverture morphologique pour nettoyer, (3) distance transform + seuillage pour identifier les centres des objets, (4) connectedComponents pour assigner un label unique par centre. On n'a donc qu'un marqueur par objet (au lieu de centaines de minima), et le watershed sépare proprement les objets contigus.

Q8 : « Pourquoi CLAHE plutôt qu'une égalisation d'histogramme standard ? »

Réponse : L'égalisation standard est **globale** : elle étire l'histogramme entier sur [0, 255]. Sur une image d'endoscopie avec 90% de pixels sombres et quelques reflets brillants, l'égalisation globale amplifie les reflets et produit une image très contrastée et bruitée. CLAHE travaille **localement** (par tuiles) et **limite** l'amplification (clip_limit), ce qui rehausse le contraste dans les zones sombres sans exploser les reflets.

Q9 : « Comment vous gérez les images DICOM vs PNG ? »

Réponse : Le module `io.py` détecte le format par l'extension du fichier. Pour DICOM, on utilise `pydicom` pour extraire le `pixel_array` et appliquer le RescaleSlope/RescaleIntercept (transformation de Hounsfield pour les CT). Pour NIfTI, on utilise `nibabel` et on extrait la coupe médiane du volume 3D. Tout est ensuite normalisé en [0, 255] uint8 et sauvé en PNG. Le frontend ne voit que des PNG — l'abstraction est au niveau du backend.

Q10 : « Si on vous donne un dataset de scanners thoraciques, quels paramètres changeriez-vous ? »

Réponse : (1) **Prétraitement** : pas de CLAHE (le CT a un bon contraste natif), mais normalisation en fenêtre pulmonaire (window/level) car les valeurs de Hounsfield ont une plage de ~4000. (2) **Architecture** : possible passage en 3D (les coupes CT sont contiguës et la tumeur s'étend sur plusieurs coupes). (3) **Perte** : garder Dice+BCE mais peut-être ajouter un terme de focal loss si les tumeurs sont très petites. (4) **Augmentation** : adapter les transformations (pas d'elastic transform car le thorax est rigide, mais rotation et scale). (5) **Dataset** : probablement plus grand (le thorax est une zone très étudiée), donc on peut se permettre un modèle plus complexe.

Q11 : « Votre modèle fonctionne-t-il sur une image non vue d'un autre hôpital ? »

Réponse : Probablement pas directement, à cause du **domain shift** (décalage de domaine). Chaque centre hospitalier utilise des endoscopes/IRM différents, des protocoles d'acquisition différents, des patients avec des caractéristiques démographiques différentes. Le modèle a appris les biais de son dataset d'entraînement. Solutions : (1) augmentation agressive pour simuler la variabilité, (2) normalisation Z-score/CLAHE pour réduire les différences inter-centres, (3) fine-tuning sur quelques images du nouveau centre (transfer learning), (4) techniques de domain adaptation (CycleGAN, adversarial training).

Q12 : « Comment vous mesureriez la robustesse de votre modèle ? »

Réponse : (1) **Cross-validation** : k-fold sur le dataset (5-fold par exemple) pour mesurer la variance du Dice. (2) **Test sur données externes** : un dataset d'un autre centre jamais vu en entraînement. (3) **Perturbations** : ajouter du bruit gaussien, changer la luminosité, flouter, et mesurer la chute de Dice. (4) **Analyse d'échec** : examiner les cas où Dice < 0.5 et identifier les patterns communs (petites lésions ? forte incertitude ?). Notre module MC Dropout fournit déjà le point (4) : la corrélation incertitude-erreur est une mesure de robustesse.

Q13 : « Pourquoi la Dice loss est-elle non-convexe et quels problèmes ça pose ? »

Réponse : La Dice loss contient un quotient $\frac{2 \sum y \hat{y}}{\sum y + \sum \hat{y}}$. Ce quotient n'est pas convexe car le numérateur et le dénominateur dépendent tous les deux des prédictions $\hat{y}$, créant un paysage de perte avec des minima locaux. Problème : le gradient peut être instable, surtout quand les deux termes du dénominateur sont proches de zéro (petits objets). C'est pour ça qu'on combine avec la BCE (qui est convexe) : la BCE stabilise le gradient dans les zones où la Dice loss est instable.

Q14 : « Qu'est-ce qu'un faux positif critique dans votre contexte ? »

Réponse : En endoscopie, un faux positif = le modèle détecte un polype qui n'existe pas. Le clinicien pourrait recommander une biopsie inutile (coûteux, risque d'infection, anxiété du patient). Un faux négatif = le modèle rate un vrai polype. Conséquence : le polype peut évoluer en cancer. **Les deux sont critiques**, mais le faux négatif est généralement considéré plus grave (manquer un cancer). C'est pourquoi on veut un rappel élevé, et le module d'incertitude aide en signalant les zones « à vérifier ».

Q15 : « Vous avez combien de paramètres dans votre U-Net ? Est-ce raisonnable ? »

Réponse : Environ **1.9 million de paramètres** (16→32→64→128→256 filtres). C'est raisonnable pour un dataset de 100 images grâce au Dropout et à l'augmentation. Un ResNet-50 a 25M paramètres — notre modèle est 13× plus petit. Pour comparaison, le U-Net original (Ronneberger, 2015) avait ~31M paramètres (64→128→256→512→1024), ce qui serait excessif pour 100 images.

Q16 : « Comment déployer ça en production ? »

Réponse : (1) **Docker Compose** déjà fonctionnel (backend + frontend + nginx). (2) Ajouter un reverse proxy HTTPS (Let's Encrypt). (3) Stocker les modèles dans un volume persistant ou un bucket S3. (4) Ajouter un worker GPU si le temps d'inférence CPU est trop long (TensorFlow supporte GPU nativement). (5) Ajouter un système d'authentification (pas implémenté car hors scope). (6) Ajouter la persistance des résultats (base de données au lieu du stockage temporaire en mémoire). (7) Validation réglementaire (norme CE, FDA) pour usage clinique — ce n'est pas un outil médical certifié.

Q17 : « Pourquoi N4 bias correction s'applique à l'IRM et pas à l'endoscopie ? »

Réponse : Le biais de champ (bias field) est un artefact physique de l'IRM causé par les inhomogénéités du champ magnétique B1 (radiofréquence). Il crée une variation lente et douce des intensités. Ce phénomène **n'existe pas** en endoscopie car l'endoscope utilise une caméra optique avec un éclairage LED — les artéfacts d'éclairage en endoscopie sont des reflets spéculaires (ponctuels et brillants), pas un biais multiplicatif lent. Appliquer N4 sur une image d'endoscopie déformerait les intensités sans corriger les vrais artefacts.

Q18 : « Quelle est la différence entre segmentation sémantique et segmentation d'instance ? »

Réponse : La segmentation **sémantique** classifie chaque pixel par catégorie (fond, tumeur, vaisseau...) mais ne distingue pas les instances : deux tumeurs adjacentes ont le même label. La segmentation **d'instance** distingue chaque objet individuel (tumeur 1 vs tumeur 2). Notre U-Net fait de la segmentation sémantique binaire (fond vs premier plan). Pour de la segmentation d'instance, il faudrait un modèle comme Mask R-CNN.

Q19 : « Pourquoi la segmentation médicale est-elle plus difficile que la segmentation d'objets naturels ? »

Réponse : (1) **Faible contraste** : les tissus biologiques ont des intensités très proches (contrairement à un chat sur un fond blanc). (2) **Frontières floues** : les bords tumoraux infiltrent les tissus sains progressivement. (3) **Variabilité anatomique** : chaque patient est différent. (4) **Bruit d'acquisition** : artefacts IRM, reflets endoscopiques. (5) **Données rares et coûteuses** : l'annotation nécessite un radiologue expert (vs n'importe qui peut annoter un chat). (6) **Conséquences critiques** : une erreur peut coûter une vie.

Q20 : « Si on vous demande de passer en 3D, comment vous feriez ? »

Réponse : (1) Remplacer les `Conv2D` par des `Conv3D`, les `MaxPooling2D` par `MaxPooling3D`, et les `Conv2DTranspose` par `Conv3DTranspose`. (2) L'entrée passe de (256, 256, 1) à (D, 256, 256, 1) où D = nombre de coupes. (3) Problème : la mémoire GPU explose (un volume 3D de 128×256×256 avec 256 filtres ne tient pas en mémoire). Solutions : (a) patchs 3D (traiter des sous-volumes), (b) architecture 2.5D (3 coupes adjacentes en entrée), (c) réduire les filtres. (4) Le MC Dropout s'applique identiquement en 3D.

---

10. Limites honnêtes du projet

Limite	Impact	Perspective d'amélioration
Validation sur seulement 10 images	Les résultats statistiques (Pearson, rejet sélectif) sont indicatifs mais pas robustes. Une corrélation sur 10 points peut être un artefact.	Utiliser un dataset plus grand (Kvasir-SEG : 1000 images, CVC-ClinicDB : 612 images).
Pas de split par patient	Si un patient a plusieurs images, certaines peuvent être en train et d'autres en val → le modèle a « vu » le patient. Data leakage.	Garantir qu'aucun patient n'est dans les deux sets (group split).
Pas de test set indépendant	On évalue sur les mêmes données que la validation → optimisme. Pas de mesure de généralisation.	Réserver 20% des données comme test set, jamais touché pendant le développement.
U-Net 2D alors que l'IRM est 3D	On perd l'information inter-coupes (une tumeur s'étend sur 10 coupes mais on traite chaque coupe indépendamment).	Passer en 2.5D (3 coupes en entrée) ou en 3D complet.
MC Dropout ≠ vraie Bayesian NN	MC Dropout approxime une distribution a posteriori par une distribution variationnelle. L'incertitude est sous-estimée par rapport à un vrai modèle bayésien (HMC, Laplace approximation).	Comparer avec des deep ensembles (plus coûteux mais meilleures incertitudes) ou une approximation de Laplace (post-hoc, pas besoin de ré-entraîner).
Aucune validation clinique	Ce n'est pas un dispositif médical. Les performances n'ont pas été évaluées par un clinicien.	Étude clinique avec des radiologues pour mesurer l'apport de l'incertitude dans la décision.
Pas comparé à nnU-Net ou SAM	On ne sait pas comment notre U-Net simple se situe par rapport à l'état de l'art.	Benchmarker sur les mêmes données avec nnU-Net (segmentation spécialisée) et SAM (modèle fondation).
Un seul type d'incertitude	MC Dropout capture l'épistémique, pas l'aléatoire.	Ajouter une branche de prédiction de variance (heteroscedastic uncertainty) pour l'aléatoire.

11. Glossaire

Terme	Définition
BCE (Binary Cross-Entropy)	Fonction de perte standard pour la classification binaire pixel par pixel. Mesure la distance entre la probabilité prédite et le label vrai (0 ou 1).
CLAHE	Contrast Limited Adaptive Histogram Equalization. Amélioration de contraste locale avec limitation de l'amplification.
Chan-Vese	Méthode de segmentation par contours actifs sans bord (level-set). Minimise une énergie basée sur les moyennes d'intensité intérieure et extérieure.
Conv2DTranspose	Convolution transposée (ou déconvolution). Opération qui augmente la résolution spatiale — inverse du pooling. Utilisée dans le décodeur du U-Net.
CORS	Cross-Origin Resource Sharing. Mécanisme HTTP qui permet à un frontend sur un domaine d'appeler une API sur un autre domaine.
DICOM	Digital Imaging and Communications in Medicine. Format standard des images médicales, contenant les pixels ET les métadonnées patient/acquisition.
Dice	Coefficient de similarité entre deux ensembles. $2
Domain shift	Décalage entre la distribution des données d'entraînement et celle des données de test (autre scanner, autre hôpital, autre population).
Dropout	Technique de régularisation qui désactive aléatoirement des neurones pendant l'entraînement pour éviter l'overfitting.
EarlyStopping	Callback d'entraînement qui arrête le processus quand la métrique surveillée ne s'améliore plus.
Entropie	Mesure de l'incertitude d'une distribution de probabilité. Maximale quand tous les événements sont équiprobables.
Épistémique	Relatif à la connaissance. L'incertitude épistémique est liée au manque de données d'entraînement et peut être réduite avec plus de données.
FastAPI	Framework web Python asynchrone, avec validation automatique des données et génération de documentation Swagger.
Hausdorff	Distance de Hausdorff : distance maximale entre un point d'un ensemble et le point le plus proche de l'autre ensemble. Mesure la pire erreur de contour. Non implémentée dans ce projet mais classique en segmentation.
he_normal	Initialisation de He (2015). Poids tirés de $\mathcal{N}(0, \sqrt{2/n_{in}})$. Conçue pour ReLU.

Terme	Définition
IoU	Intersection over Union (indice de Jaccard). $
Keras	API haut niveau pour TensorFlow. Permet de définir des modèles de deep learning en quelques lignes.
Level-set	Représentation implicite d'un contour par une fonction ϕ : le contour est la courbe $\phi = 0$. Permet de gérer les changements de topologie.
LGG	Low-Grade Glioma. Type de tumeur cérébrale. Le dataset LGG MRI Segmentation (Kaggle) contient des IRM avec segmentation manuelle.
MC Dropout	Monte Carlo Dropout. Technique qui garde le Dropout actif à l'inférence et effectue N passes stochastiques pour estimer l'incertitude.
Mean Shift	Algorithme de clustering non-paramétrique qui fait converger les points vers les modes de la distribution de densité locale.
MSE	Mean Squared Error. Moyenne des carrés des différences pixel à pixel.
NIfTI	Neuroimaging Informatics Technology Initiative. Format de fichier pour les volumes 3D en neuroimagerie (.nii, .nii.gz).
NMI	Normalized Mutual Information. Mesure de dépendance statistique normalisée entre deux images.
N4 bias	Algorithme de correction du biais de champ magnétique en IRM (Tustison et al., 2010). Modélise le biais par une B-spline et le corrige itérativement.
Otsu	Méthode de seuillage automatique qui maximise la variance inter-classes. Fonctionne bien sur les histogrammes bimodaux.
Overfitting	Surapprentissage. Le modèle mémorise les données d'entraînement au lieu de généraliser. Se manifeste par un bon score en train et un mauvais en validation.
Pearson	Coefficient de corrélation linéaire entre deux variables. Plage [-1, 1].
Region growing	Segmentation par croissance de région depuis un point graine. Propage aux voisins de même intensité.
ReLU	Rectified Linear Unit. $f(x) = \max(0, x)$. Activation non-linéaire standard des CNN.
Sigmoid	

Terme	Définition
	$\sigma(x) = 1/(1+e^{-x})$. Fonction qui comprime toute valeur réelle dans [0, 1]. Utilisée en sortie pour la probabilité de segmentation.
SimpleITK	Bibliothèque Python pour le traitement d'images médicales. Fournit N4BiasFieldCorrection, des opérations de recalage, etc.
Skip connection	Connexion directe entre une couche de l'encodeur et la couche symétrique du décodeur dans un U-Net. Transmet les détails fins.
TensorFlow	Framework de deep learning de Google. Keras est son API haut niveau.
U-Net	Architecture de réseau pour la segmentation d'images médicales (Ronneberger et al., 2015). Forme en U : encodeur-décodeur avec skip connections.
Watershed	Segmentation par inondation topographique. Nécessite des marqueurs pour éviter la sur-segmentation.
Zustand	Gestionnaire d'état minimaliste pour React. Remplace Redux avec beaucoup moins de boilerplate.

---

12. Checklist mentale pour le jour J

Mémorise ces 10 phrases pour présenter le projet en 1 minute :

1. **Ce que ça fait** : « C'est une plateforme web qui permet d'importer une image médicale et de la segmenter automatiquement par des méthodes classiques ou par deep learning, avec une mesure de confiance intégrée. »
2. **L'architecture** : « Le backend est en FastAPI/Python avec OpenCV et TensorFlow, le frontend en React/TypeScript — ils communiquent par API REST en échangeant des IDs d'images, pas des fichiers binaires. »
3. **Les méthodes classiques (1/2)** : « Côté prétraitement, on a 9 filtres dont CLAHE pour l'endoscopie et N4 pour l'IRM, plus les opérations morphologiques et les détecteurs de contours. »
4. **Les méthodes classiques (2/2)** : « Côté segmentation conventionnelle, 7 méthodes sont implémentées — Otsu, watershed avec marqueurs construits par distance transform, Chan-Vese, K-means, Mean Shift, et region growing — toutes évaluées par Dice et IoU. »
5. **Le DL (1/2)** : « Le U-Net suit exactement l'architecture du TP4 avec 5 blocs encodeur, Dropout progressif de 0.1 à 0.3, et skip connections — mais

l'entraînement est amélioré avec une perte combinée Dice+BCE et de l'augmentation albumentations. »

6. **Le DL (2/2)** : « Ces améliorations corrigent les deux problèmes du TP4 : le Dice à zéro (résolu par la Dice loss qui force la détection du foreground minoritaire) et l'overfitting sur 100 images (résolu par l'augmentation et l'early stopping). »
7. **MC Dropout (1/2)** : « L'extension recherche utilise le Monte Carlo Dropout : on garde le Dropout actif à l'inférence et on fait 30 passes pour obtenir une distribution de prédictions, dont la variance est l'incertitude épistémique du modèle. »
8. **MC Dropout (2/2)** : « Le notebook montre que l'incertitude est corrélée à l'erreur de segmentation, et que rejeter les 20% de prédictions les moins confiantes améliore significativement le Dice — c'est un scénario de triage clinique. »
9. **Les limites** : « La principale limite est la taille du dataset — 100 images, 10 en validation, pas de test set indépendant — ce qui limite la robustesse des conclusions statistiques. »
10. **Ce que j'ai appris** : « Ce projet m'a appris à aller au-delà de la prédiction binaire : en imagerie médicale, savoir *quand* le modèle se trompe est aussi important que la prédiction elle-même. »